



ISIMA

Institut Supérieur d'Informatique, de Modélisation et de leurs Applications
Université Clermont Auvergne (UCA)

Rapport de Projet IA :

Réalisé par :

Rzeigui Ahmed
Masfour Yassir
Nasri Ayoub
Mohamed Amine Rannen

Année universitaire : *2025 – 2026*

Table des matières

Résumé	5
Remerciements	5
1 Introduction générale	6
1.1 Contexte du projet	6
1.2 Problématique et objectifs	6
1.3 Périmètre et hypothèses	7
1.4 Organisation du rapport	7
2 Gestion de projet et tâches réalisées	8
2.1 Méthodologie de travail	8
2.2 Outils collaboratifs et gestion de versions (GitLab)	8
2.3 Répartition des tâches	8
2.4 Planning et jalons	9
2.5 Risques et stratégies d'atténuation	9
2.6 Bilan de gestion de projet	9
3 Tuteur IA basé sur RAG	10
3.1 Introduction de la partie	10
3.2 Spécifications et besoins	10
3.2.1 Fonctionnalités attendues	10
3.2.2 Contraintes (temps, données, sécurité, coût)	10
3.3 Données et base documentaire	10
3.3.1 Sources des documents	10
3.3.2 Nettoyage et préparation	11
3.3.3 Stratégie de découpage (chunking large)	11
3.4 Conception et fonctionnement du RAG	12
3.5 Choix techniques et justification	12
3.5.1 Justification globale de l'approche RAG	12
3.5.2 Choix des embeddings	12
3.5.3 Choix du <i>vector store</i>	13
3.5.4 Choix du <i>retriever</i> et du <i>reranking</i>	13
3.5.5 Choix du LLM et du <i>prompting</i>	13
3.6 Rôle du tuteur IA dans la conception de l'IA apprenante	14
3.7 Implémentation	14
3.7.1 Stack technique	14
3.7.2 Organisation logicielle	15
3.8 Évaluation du tuteur	15

3.8.1	Illustration du fonctionnement du RAG	16
3.8.2	Critères de pertinence	16
3.8.3	Évaluation quantitative des réponses	17
3.8.4	Propositions et améliorations	17
4	IA apprenante : CNN, MLP et Transfer Learning	19
4.1	Introduction de la partie	19
4.2	Données et prétraitement	19
4.2.1	Dataset : description et exploration	19
4.2.2	Prétraitements (normalisation, augmentation)	21
4.2.3	Découpage train / validation / test	21
4.3	Modèles Deep Learning	22
4.3.1	Perceptron multicouche (MLP)	22
4.3.2	Approche CNN	23
4.3.3	Transfer Learning	24
4.3.4	Comparaison Globale des Modèles	25
4.4	Limites et améliorations	27
4.4.1	Augmentation de Données (Data Augmentation)	27
4.4.2	Stratégie d'Arrêt (Eco-Conception)	27
4.4.3	Pistes d'amélioration (Edge AI)	27
5	RSE et Impact Environnemental	28
5.1	Mesures énérgitque pour les LLMs du Tuteur IA	28
5.2	Méthodologie et environnement	28
5.2.1	Matériel utilisé	28
5.3	Résultats	29
5.3.1	Émissions CO ₂ et énergie consommée	29
5.3.2	Décomposition de l'énergie par composant	29
5.3.3	Taux d'utilisation des composants	30
5.4	Visualisations	30
5.4.1	Émissions CO ₂ par modèle	30
5.4.2	Répartition de l'énergie par composant	31
5.4.3	Durée d'exécution comparative	31
5.4.4	Taux d'utilisation des composants	31
5.5	Analyse et interprétation	32
5.5.1	Classement global	32
5.5.2	Le cas particulier de Qwen2.5-Coder-0.5B	32
5.5.3	Efficacité par milliard de paramètres	32
5.6	Conclusions	33
5.7	Analyse Comparative IA apprenante : Performance vs Sobriété	34
5.8	Interprétation et Conclusion RSE	34
6	Conclusion générale et perspectives	35
	Bibliographie	36

Table des figures

3.1	Réponse du tuteur IA avec affichage du contexte récupéré par le mécanisme RAG	16
3.2	Suite de la réponse générée par le tuteur IA	16
4.1	Exemples d'images issues du dataset (une image par classe).	20
4.2	Apprentissage MLP : Convergence difficile et instable.	25
4.3	Apprentissage CNN : Progression fluide et généralisation correcte.	25
4.4	Évolution de perte transfert learning 25 epochs	25
4.5	Évolution de l'accuracy transfert learning 25 epochs	25
4.6	Banana : Concentration sur la forme du fruit.	26
4.7	Pizza : La texture de la garniture active le modèle.	26
4.8	Sushi : Focalisation sur le riz et le poisson.	26
4.9	Tomate : La rondeur et la couleur sont déterminantes.	26
4.10	Visualisation Grad-CAM. Les zones rouges indiquent l'attention du modèle.	26
4.11	Visualisation de la Data Augmentation : Variations générées pour l'entraînement.	27
5.1	Émissions CO ₂ par modèle ($\times 10^{-5}$ kg CO ₂ eq)	30
5.2	Répartition de l'énergie consommée par composant	31
5.3	Durée d'exécution de la phase de génération	31
5.4	Taux d'utilisation moyen des composants par modèle	31
5.5	Paradoxe énergétique : Le modèle le plus performant n'est pas forcément le plus polluant à l'usage.	34

Liste des tableaux

3.1	Principales sources documentaires utilisées pour le tuteur IA	11
3.2	Évaluation de la pertinence des réponses du tuteur IA	17
4.1	Répartition des données par ensemble (train/validation/test).	22
4.2	Synthèse comparative des performances sur l'ensemble de validation (avg scores)	26
5.1	Métriques principales – Phase de génération (triées par émissions croissantes)	29
5.2	Répartition de l'énergie par composant (en Wh)	30
5.3	Taux d'utilisation moyen des composants pendant la génération	30
5.4	Efficacité carbone par milliard de paramètres	32

Remerciements

Nous souhaitons exprimer nos remerciements à l'équipe pédagogique de l'ISIMA et de l'Université Clermont Auvergne pour l'encadrement, les enseignements et les ressources mises à disposition tout au long de ce projet.

Nous remercions également les enseignants et intervenants pour leurs conseils méthodologiques et techniques, ainsi que pour l'attention portée aux enjeux de qualité logicielle et de responsabilité environnementale dans le cadre de ce projet.

Enfin, nous tenons à remercier l'ensemble des membres de l'équipe pour leur investissement, leur collaboration et la complémentarité de leurs compétences, qui ont permis la réussite de ce travail collectif.

Chapitre 1

Introduction générale

1.1 Contexte du projet

L'intelligence artificielle occupe aujourd'hui une place centrale dans le développement des systèmes logiciels modernes. Les techniques de *deep learning* ont notamment permis des avancées significatives dans des domaines tels que la vision par ordinateur, la reconnaissance d'images ou encore le traitement automatique du langage naturel.

Parallèlement, l'émergence des modèles de langage de grande taille (*Large Language Models*, LLM) et des approches de type *Retrieval-Augmented Generation* (RAG) a ouvert de nouvelles perspectives dans l'assistance au développement logiciel. Ces agents intelligents sont capables d'interroger des bases documentaires, d'expliquer du code, d'identifier des erreurs et de proposer des optimisations adaptées à un contexte donné.

Cependant, l'utilisation croissante de modèles de deep learning soulève des enjeux énergétiques et environnementaux majeurs. L'entraînement, le *fine-tuning* et l'inférence de ces modèles impliquent une consommation importante de ressources de calcul. Il devient ainsi indispensable d'intégrer des démarches d'écoconception et d'évaluation de l'impact environnemental dès la phase de conception des solutions logicielles.

Ce projet s'inscrit dans cette dynamique, en proposant une approche combinant intelligence artificielle générative, deep learning, ingénierie logicielle et prise en compte des enjeux énergétiques.

1.2 Problématique et objectifs

La conception de modèles de deep learning performants repose sur des choix techniques complexes, tant au niveau des architectures que des hyperparamètres et des outils logiciels utilisés. Dans ce contexte, l'assistance par un agent IA tuteur apparaît comme une solution pertinente pour accompagner le développeur tout au long du processus de conception, de développement et d'évaluation d'un logiciel intégrant de l'intelligence artificielle.

La problématique centrale de ce projet peut être formulée de la manière suivante :

Comment concevoir un agent IA tuteur capable d'assister efficacement le développement d'un logiciel intégrant du deep learning, tout en conciliant performance, qualité logicielle et sobriété énergétique ?

Afin de répondre à cette problématique, les objectifs du projet sont les suivants :

- concevoir un agent IA tuteur basé sur une approche RAG, capable d'assister le développement en langage Python ;

- utiliser ce tuteur pour accompagner la conception d’une IA apprenante dédiée à une tâche de classification d’images ;
- implémenter et comparer plusieurs modèles de deep learning, incluant un perceptron multicouche (MLP), un réseau de neurones convolutifs (CNN) et une approche de *transfer learning* ou de *fine-tuning* ;
- évaluer les performances des modèles à l’aide de métriques adaptées, telles que l’accuracy et le F1-score ;
- mesurer et analyser l’impact énergétique et environnemental des différentes étapes du projet à l’aide d’outils tels que CodeCarbon.

1.3 Périmètre et hypothèses

Le périmètre de ce projet se limite à la conception d’un agent IA tuteur et d’une IA apprenante dédiés à une tâche de classification d’images. Le tuteur IA est conçu pour assister le développement logiciel en langage Python, en ciblant principalement les bibliothèques `scikit-learn`, `TensorFlow` et `PyTorch`.

L’IA apprenante est appliquée à un problème de vision par ordinateur, à savoir la classification d’images issues du jeu de données *banana-sushi*, composé de classes non apprises par les modèles pré-entraînés standards.

Les hypothèses suivantes sont retenues :

- les ressources matérielles sont limitées à un environnement de calcul académique ;
- les performances sont évaluées dans un cadre expérimental et comparatif ;
- les mesures énergétiques issues de CodeCarbon constituent des estimations exploitables pour une analyse relative.

1.4 Organisation du rapport

Le rapport est structuré en plusieurs chapitres. Le chapitre 2 présente l’organisation du projet (répartition des tâches, outils, planning). Le chapitre 3 détaille la conception du tuteur IA basé sur RAG et son évaluation. Le chapitre 4 présente l’IA apprenante (MLP, CNN, transfer learning) et l’analyse des performances. Le chapitre 5 discute l’impact énergétique et environnemental (CodeCarbon) ainsi que la réflexion RSE. Enfin, le chapitre 6 conclut et propose des perspectives.

Chapitre 2

Gestion de projet et tâches réalisées

2.1 Méthodologie de travail

Le projet a été réalisé par une équipe de quatre personnes. L'organisation du travail a été pensée de manière à permettre un développement parallèle des deux axes principaux du projet : la conception de l'agent IA tuteur et celle de l'IA apprenante.

Une approche itérative a été adoptée, avec des cycles courts de conception, d'implémentation et de validation. Des échanges réguliers au sein de l'équipe ont permis de coordonner les avancées et d'assurer la cohérence globale de la solution développée.

2.2 Outils collaboratifs et gestion de versions (GitLab)

La gestion du code source a été assurée à l'aide de la plateforme GitLab, qui a servi de support central pour le développement collaboratif. Le dépôt GitLab a permis de structurer le projet, de suivre l'évolution du code et de faciliter l'intégration des différentes contributions.

L'utilisation de branches dédiées a permis de séparer le développement des fonctionnalités liées au tuteur IA et celles relatives à l'IA apprenante, avant leur fusion progressive dans la branche principale. Les commits réguliers ont contribué à une meilleure traçabilité des modifications. Les contributions individuelles sont consultables via l'historique des commits.

2.3 Répartition des tâches

La répartition des tâches a été réalisée en fonction des deux volets principaux du projet. Deux membres de l'équipe ont été principalement chargés de la conception et de l'implémentation de l'agent IA tuteur basé sur une approche RAG. Leurs tâches ont notamment porté sur la constitution de la base documentaire, la mise en place du pipeline de récupération d'informations et la génération des réponses.

Les deux autres membres se sont concentrés sur la conception de l'IA apprenante dédiée à la classification d'images. Ils ont pris en charge l'implémentation des modèles de deep learning (MLP, CNN et transfer learning), leur entraînement, leur évaluation et l'analyse des résultats.

2.4 Planning et jalons

Le projet a été structuré autour de plusieurs jalons clés : (1) analyse du sujet et définition de l'architecture globale, (2) développement parallèle du tuteur IA et de l'IA apprenante, (3) évaluation des performances, analyse énergétique et rédaction des livrables. Ce découpage a permis de respecter les délais tout en assurant une progression cohérente.

2.5 Risques et stratégies d'atténuation

Plusieurs risques ont été identifiés, notamment la complexité liée à la mise en œuvre conjointe d'un agent IA tuteur et de modèles de deep learning, ainsi que les risques d'intégration (conflits de code, incompatibilités, dérives de périmètre).

Pour limiter ces risques, le travail a été découpé en sous-parties, des points de synchronisation réguliers ont été organisés, et GitLab a été utilisé pour sécuriser l'intégration (branches, merges, traçabilité).

2.6 Bilan de gestion de projet

L'organisation au sein d'une équipe de quatre personnes, associée à une répartition claire des tâches et à l'utilisation de GitLab, a permis de mener à bien le projet dans les délais impartis. Le développement parallèle a contribué à une meilleure efficacité tout en conservant une cohérence globale.

Chapitre 3

Tuteur IA basé sur RAG

3.1 Introduction de la partie

Ce chapitre présente la conception et le fonctionnement d'un agent IA tuteur destiné à assister le développement logiciel en langage Python. Le tuteur vise à expliquer des concepts, corriger des erreurs, proposer des optimisations et accompagner la documentation du code. Afin de limiter les hallucinations et d'ancrer ses réponses sur des sources fiables, il repose sur une architecture de type Retrieval-Augmented Generation (RAG).

3.2 Spécifications et besoins

3.2.1 Fonctionnalités attendues

- interroger une base documentaire pédagogique ;
- expliquer des concepts liés au machine learning et au deep learning ;
- assister la correction et l'amélioration de code Python ;
- cibler les bibliothèques `scikit-learn`, `TensorFlow` et `PyTorch` ;
- adopter un comportement pédagogique structuré.

3.2.2 Contraintes (temps, données, sécurité, coût)

Le projet s'inscrit dans un cadre académique avec des ressources matérielles limitées. Le temps de réponse doit rester compatible avec une utilisation interactive. Le tuteur exploite uniquement des données non sensibles issues de la base documentaire définie. L'impact énergétique de la solution doit être mesuré.

3.3 Données et base documentaire

3.3.1 Sources des documents

La base documentaire utilisée par le tuteur IA repose sur un ensemble de ressources pédagogiques et techniques issues principalement de supports universitaires et de documentations officielles. Ces documents ont été sélectionnés afin de couvrir à la fois les fondements théoriques du deep learning et des exemples pratiques d'implémentation en Python.

Les sources exploitées se répartissent en plusieurs catégories :

- **Supports de cours universitaires** portant sur les réseaux de neurones, les réseaux de neurones convolutifs (CNN), les autoencodeurs et les transformers, issus de formations de niveau Master ;
- **Documents pédagogiques sur l'apprentissage par transfert**, expliquant les principes du *transfer learning* et du *fine-tuning* dans des cadres applicatifs concrets ;
- **Documentation officielle** de bibliothèques Python utilisées dans le projet, notamment `scikit-learn`, `TensorFlow` et `PyTorch` ;
- **Ressources académiques complémentaires** provenant d'universités françaises (Aix-Marseille Université, Université Lyon 2) et de contenus pédagogiques spécialisés (par exemple les supports de Joel Kaazadi).

TABLE 3.1 – Principales sources documentaires utilisées pour le tuteur IA

Type	Origine	Contenu principal
Support de cours	Univ. Batna 2	Réseaux de neurones convolutifs (CNN), couches, architectures
Support de cours	Université	Autoencodeurs et apprentissage non supervisé
Support de cours	Université	Transformers et modèles de langage
Document pédagogique	TensorFlow	Apprentissage par transfert et fine-tuning
Documentation officielle	scikit-learn	Modèles ML, métriques, pipelines
Documentation officielle	PyTorch	Implémentation de réseaux neuronaux profonds
Ressource académique	Aix-Marseille / Lyon 2	Vision par ordinateur et deep learning
Contenu pédagogique	Joel Kaazadi	Explications pratiques et vulgarisation DL

Ces documents constituent le socle de connaissances interrogé par le système RAG. Ils permettent au tuteur IA de fournir des réponses contextualisées, techniquement fondées et adaptées aux problématiques rencontrées lors du développement des modèles MLP, CNN et de l'approche par transfer learning.

3.3.2 Nettoyage et préparation

Les documents sont nettoyés afin de supprimer les éléments non pertinents (en-têtes, pieds de page, bruit textuel) et de normaliser le contenu.

3.3.3 Stratégie de découpage (chunking large)

Les documents sont découpés en segments de taille relativement importante afin de préserver un contexte suffisant pour les questions explicatives.

3.4 Conception et fonctionnement du RAG

La conception détaillée de l’agent IA tuteur est présentée dans un document dédié, référencé et joint aux livrables

3.5 Choix techniques et justification

3.5.1 Justification globale de l’approche RAG

L’utilisation d’un modèle de langage seul (*Large Language Model*, LLM) pour l’assistance au développement logiciel présente une limite majeure : la production de réponses syntaxiquement plausibles mais factuellement incorrectes, communément désignées sous le terme d’*hallucinations*. Ce phénomène est particulièrement problématique dans un contexte pédagogique et technique, où la fiabilité des explications, des raisonnements et du code généré constitue un enjeu central.

L’approche *Retrieval-Augmented Generation* (RAG) vise à pallier cette limitation en combinant deux mécanismes complémentaires :

- une phase de récupération (*retrieval*) de documents pertinents à partir d’une base documentaire externe ;
- une phase de génération conditionnée par le contexte informationnel récupéré.

Dans le cadre de ce projet, le RAG permet d’ancrer les réponses du tuteur IA sur des sources fiables, telles que des supports de cours universitaires, des documentations officielles et des contenus techniques validés. Cette approche contribue à réduire significativement le risque d’hallucinations, tout en améliorant la pertinence, la précision et la valeur pédagogique des réponses fournies.

3.5.2 Choix des embeddings

Les documents de la base documentaire ainsi que les requêtes des utilisateurs sont projetés dans un espace vectoriel à l’aide du modèle d’embeddings `all-MiniLM-L6-v2`, issu de la bibliothèque `sentence-transformers`.

Chaque document est préalablement découpé en segments (*chunks*). Pour chaque chunk, un processus de tokenisation est appliqué, consistant à associer chaque élément textuel à un identifiant de token (*token ID*) issu du vocabulaire du modèle. Ces tokens sont ensuite transformés en une représentation vectorielle dense de dimension 384, correspondant à l’espace latent appris par le modèle `SentenceTransformer`. Le vecteur obtenu constitue l’embedding final du chunk.

Ce modèle a été retenu pour plusieurs raisons :

- il offre un compromis pertinent entre qualité de représentation sémantique et coût computationnel ;
- il est particulièrement adapté aux tâches de similarité sémantique appliquées à des textes techniques et pédagogiques ;
- sa taille modérée permet une exécution locale efficace, en cohérence avec une démarche de sobriété énergétique.

Les embeddings sont générés aussi bien pour les chunks documentaires que pour les requêtes utilisateurs. La comparaison sémantique repose sur la similarité cosinus, utilisée pour estimer la pertinence des segments de texte vis-à-vis d’une requête donnée.

3.5.3 Choix du *vector store*

La base vectorielle repose sur **ChromaDB**, exploitée avec une indexation de type HNSW (*Hierarchical Navigable Small World*) et une mesure de similarité cosinus.

Ce choix se justifie par les éléments suivants :

- l'utilisation d'une structure d'indexation hiérarchique permettant de réduire la complexité de recherche, passant d'une recherche exhaustive en $\mathcal{O}(N)$ à une recherche approximative en $\mathcal{O}(\log N)$;
- la simplicité d'intégration dans un environnement Python ;
- la gestion native des métadonnées associées aux documents ;
- de bonnes performances pour la recherche approximative de plus proches voisins ;
- une compatibilité naturelle avec des architectures RAG exécutées localement.

Chaque chunk est stocké avec son embedding ainsi qu'un ensemble de métadonnées (titre, type de document, source), facilitant la traçabilité et l'analyse des informations exploitées lors de la génération.

3.5.4 Choix du *retriever* et du *reranking*

Le mécanisme de récupération de l'information repose sur une stratégie en deux étapes.

Dans un premier temps, une recherche à rappel large (*high recall*) est effectuée à partir de la base vectorielle ChromaDB. Un ensemble étendu de chunks candidats est récupéré à partir de l'embedding de la requête utilisateur. Selon le type de question détecté (définition, explication, comparaison, etc.), deux stratégies sont appliquées :

- pour les questions génériques, la requête est enrichie par l'ajout de termes explicatifs (tels que *définition*, *principe*, *fonctionnement*) afin de favoriser la récupération de contenus à forte valeur explicative ;
- pour les autres types de questions, les scores de similarité sont directement issus de la recherche vectorielle. Dans **ChromaDB**, la distance est définie comme $d = 1 - \cos(\theta)$; dans ce projet, le score de similarité est donc considéré comme $s = 1 - d$.

Dans un second temps, un mécanisme de *reranking* est appliqué à l'aide d'un *Cross-Encoder* (**ms-marco-MiniLM-L-6-v2**). La recherche vectorielle permet d'identifier rapidement un ensemble de candidats pertinents ; le cross-encoder est ensuite appliqué uniquement aux k meilleurs résultats (*top-k*) afin de les reclasser plus finement. Contrairement aux embeddings indépendants, le cross-encoder évalue directement chaque paire (requête, chunk) et produit un score de pertinence global.

Cette stratégie hybride permet :

- d'améliorer la précision des résultats finaux ;
- de sélectionner les chunks les plus pertinents avant la phase de génération ;
- de limiter le bruit contextuel transmis au modèle de langage.

3.5.5 Choix du LLM et du *prompting*

Plusieurs modèles de langage locaux ont été évalués au cours du projet, notamment :

- **meta-llama/Llama-3.2-3B-Instruct** ;
- **Qwen/Qwen2.5-Coder-0.5B-Instruct** ;

- différents modèles de la famille **Mistral**.

Les expérimentations ont montré que le modèle **Mistral-7B-Instruct** offrait le meilleur compromis entre qualité des réponses, cohérence technique et capacité à exploiter efficacement le contexte fourni par le RAG. Il a donc été retenu comme modèle principal pour la génération.

Le *prompting* repose sur une instruction explicite définissant le rôle du tuteur IA, spécialisé en Python, **scikit-learn**, **TensorFlow** et **PyTorch**. Deux situations sont distinguées :

- une génération conditionnée par le contexte RAG lorsque celui-ci est disponible et jugé pertinent ;
- une génération fondée sur les connaissances internes du modèle lorsqu’aucun contexte fiable n’est identifié.

Des paramètres de génération conservateurs ont été retenus : une température faible afin de limiter la créativité du modèle, combinée à une valeur élevée de *top-p* afin de préserver la richesse lexicale. Ce réglage favorise des réponses stables, précises et reproductibles, adaptées à un usage pédagogique et technique.

3.6 Rôle du tuteur IA dans la conception de l’IA apprenante

Le tuteur IA a été mobilisé tout au long du développement de l’IA apprenante afin d’assister les différentes étapes de conception, d’entraînement et d’évaluation des modèles.

Il a notamment permis :

- d’expliquer des erreurs d’entraînement (choix de la fonction de perte, surapprentissage, instabilités numériques) ;
- de proposer des ajustements d’hyperparamètres ;
- d’aider à la compréhension et à la mise en œuvre des architectures MLP, CNN et de transfer learning ;
- d’améliorer la lisibilité, la structuration et la documentation du code.

3.7 Implémentation

3.7.1 Stack technique

Le tuteur IA est implémenté intégralement en Python. Les principales bibliothèques utilisées sont :

- **sentence-transformers** pour la génération des embeddings ;
- **chromadb** pour la gestion de la base vectorielle ;
- **transformers** pour le chargement et l’exécution des modèles de langage ;
- **PyPDF2** pour l’extraction de texte à partir de fichiers PDF ;
- **langChain** (**RecursiveCharacterTextSplitter**) pour le découpage des documents.

3.7.2 Organisation logicielle

L’architecture logicielle est structurée en modules fonctionnels distincts :

- initialisation des composants (embeddings, base vectorielle, LLM) ;
- ingestion, nettoyage et découpage des documents ;
- recherche, scoring et reranking des chunks ;
- génération de réponses conditionnée par le contexte.

Cette organisation favorise la maintenabilité du code et facilite l’intégration de futures améliorations.

3.8 Évaluation du tuteur

L’évaluation du tuteur IA repose sur des scénarios de questions techniques couvrant le deep learning, la vision par ordinateur et les bibliothèques Python ciblées. L’objectif est d’analyser à la fois la qualité pédagogique des réponses produites et l’apport réel du mécanisme de *Retrieval-Augmented Generation* (RAG).

Les performances ont été analysées selon les critères suivants :

- la pertinence et la précision des réponses ;
- la fidélité au contexte documentaire fourni ;
- la stabilité et la cohérence des résultats.

3.8.1 Illustration du fonctionnement du RAG

À titre d'illustration, la Figure suivante présente un exemple de réponse validée par le mécanisme RAG. À la question « *comment coder un CNN pour classifier des chiens et des chats ?* », le tuteur IA fournit une réponse structurée, explicitement fondée sur le contexte récupéré, et décrivant les différentes étapes de l'implémentation.

```
Votre question: comment coder un cnn pour classifier un cnn entre des chiens et des chats ?

Recherche du contexte...

=== Contexte RAG ===
furent introduits dans la littérature par Rumelhart et al. (1986). Il a fallu attendre le papier de LeCun
et al. (1998) pour appliquer les CNNs à un problème du monde réel, ce qui a permis de les populariser.
Les CNNs sont une catégorie des réseaux artificiels composés de plusieurs couches, dont les
couches de convolution. Ces dernières appliquent un ensemble de filtres apprenables aux données
-----Next chunk-----
Par exemple, si le problème consiste à distinguer les chats des chiens, le vecteur final sera de taille 2 :
le premier élément (respectivement, le deuxième) donne la probabilité d'appartenir à la classe
"chat" (respectivement "chien"). Ainsi, le vecteur [0.9 0.1] signifie que l'image a 90% de chances de
représenter un chat.
Chapitre 2 Réseaux de neurones (Master 2 SDS) 2021
Univ-batant2 Dr. Merzougui Deep learning
-----Next chunk-----
6. Architecture d'un CNN
1. Introduction
Dans les algorithmes traditionnels de machine learning, la notion de features
(caractéristiques) en vision sont utilisées pour faire de la classification d'images. Ces méthodes
consistent à extraire les features de chaque image du jeu de données manuellement par un
expert, puis à entraîner un classifieur sur ces features. Ces techniques d'apprentissage
Confiance: 65.78%

Génération de la réponse...
```

FIGURE 3.1 – Réponse du tuteur IA avec affichage du contexte récupéré par le mécanisme RAG

```
=====
RÉPONSE
=====
Basé sur le contexte fourni:

Pour coder un CNN pour la classification entre des chiens et des chats, vous pouvez suivre ces étapes :

1. **Importez les bibliothèques nécessaires** :
'''python
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
'''

2. **Créez un modèle séquentiel** :
'''python
model = Sequential()
'''

3. **Ajoutez des couches au modèle** :
- **Convolutional Layer** : Applique des filtres aux données pour détecter des caractéristiques spécifiques.
'''python
model.add(Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape=(img_rows, img_cols, 1)))
'''
- **Max Pooling** : Réduit la dimensionnalité des données après chaque convolution.
'''python
model.add(MaxPooling2D(pool_size=(2, 2)))
'''
- **Flatten** : Convertit les données 2D en 1D pour être entraînées par les couches suivantes.
'''python
model.add(Flatten())
'''
```

FIGURE 3.2 – Suite de la réponse générée par le tuteur IA

3.8.2 Critères de pertinence

Une réponse produite par le tuteur IA est classée selon les critères suivants :

- **Pertinente** : la réponse est correcte, cohérente techniquement et exploite un contexte RAG lorsqu'il est disponible ;
- **Partiellement pertinente** : la réponse est globalement correcte mais incomplète, approximative ou faiblement ancrée dans le contexte ;
- **Non pertinente** : la réponse est incorrecte, hors-sujet ou trop vague pour un usage pédagogique.

3.8.3 Évaluation quantitative des réponses

TABLE 3.2 – Évaluation de la pertinence des réponses du tuteur IA

#	Question	RAG	Confiance	Mode	Pertinence
1	C'est quoi le transfer learning ?	Non	–	Général	Pertinente
2	Définition d'un CNN	Oui	74.70%	RAG	Pertinente
3	À quoi sert la normalisation des données ?	Non	–	Général	Pertinente
4	Qu'est-ce que la fonction de perte ?	Non	–	Général	Pertinente
5	Définition de l'overfitting	Non	–	Général	Partielle
6	Qu'est-ce qu'un embedding ?	Non	–	Général	Partielle
7	C'est quoi ImageNet ?	Non	–	Général	Partielle
8	Comment fonctionne un CNN ?	Oui	53.99%	RAG	Pertinente
9	Pourquoi utiliser la data augmentation ?	Oui	63.92%	RAG	Pertinente
10	Pourquoi figer les couches en TL ?	Oui	69.42%	RAG	Pertinente
11	Comment fonctionne ResNet ?	Oui	70%	RAG	Pertinente
12	Comment réduire l'overfitting ?	Oui	68%	RAG	Pertinente
13	CNN vs Transfer Learning	Oui	66%	RAG	Pertinente
14	Coder une CNN simple	Oui	60%	RAG	Pertinente

Score global de pertinence. Sur les 14 questions évaluées :

- 11 réponses sont jugées **pertinentes**,
- 3 réponses sont jugées **partiellement pertinentes**,
- aucune réponse n'est jugée non pertinente.

Cela correspond à un score global de :

11 / 14 réponses pleinement pertinentes ($\approx 79\%$)

Ces résultats indiquent que le tuteur IA fournit des réponses majoritairement fiables et pédagogiques, avec une amélioration notable de la qualité lorsque le mécanisme RAG est activé.

3.8.4 Propositions et améliorations

Bien que l'architecture RAG mise en place fournisse des résultats satisfaisants en termes de pertinence et de fiabilité des réponses, plusieurs axes d'amélioration peuvent être envisagés afin d'enrichir les capacités du système, d'élargir les types de sources exploitables et d'améliorer l'expérience utilisateur.

Intégration de l'OCR pour les documents multimodaux : Une première amélioration majeure consisterait à intégrer un module de reconnaissance optique de caractères (*Optical Character Recognition*, OCR) au pipeline RAG. En effet, une partie significative des documents pédagogiques et techniques (supports de cours, livres numérisés, articles scientifiques) contient des informations sous forme d'images intégrées dans des fichiers PDF (schémas, figures annotées, captures de code, tableaux scannés).

L'intégration d'un moteur OCR, tel que `Tesseract OCR` ou `Mistral OCR`, permettrait d'extraire automatiquement le texte présent dans ces images. Le contenu ainsi extrait pourrait ensuite être :

- nettoyé et structuré ;
- découpé en chunks cohérents ;
- projeté dans l'espace vectoriel à l'aide du modèle d'embeddings existant ;
- indexé dans la base vectorielle au même titre que les documents textuels.

Cette extension transformerait le système actuel en un RAG multimodal partiel, capable d'exploiter des informations visuelles encapsulées dans des documents PDF.

Exploitation des schémas et figures techniques : En complément de l'OCR, une évolution plus avancée consisterait à exploiter explicitement les figures et schémas techniques présents dans les documents. Après extraction du texte via OCR, des métadonnées supplémentaires (type de figure, contexte du cours, légende associée) pourraient être ajoutées aux chunks afin d'améliorer la récupération contextuelle.

Cette approche serait particulièrement pertinente pour des domaines tels que le deep learning, où les diagrammes d'architecture, les pipelines de traitement ou les schémas mathématiques jouent un rôle central dans la compréhension des concepts.

Amélioration du *chunking* et du routage des requêtes : Une autre piste d'amélioration concerne la stratégie de découpage des documents et le routage des requêtes. Actuellement, le chunking repose principalement sur la taille du texte. Une segmentation sémantique plus fine (basée sur les titres, sections ou types de contenu) permettrait de créer des chunks plus cohérents sur le plan conceptuel.

De plus, un mécanisme de *query routing* pourrait être introduit afin d'orienter la requête utilisateur vers des sous-bases spécialisées (cours, documentation officielle, exemples de code, contenus OCR), améliorant ainsi la précision et la pertinence des documents récupérés.

Extension vers un RAG multimodal complet : À plus long terme, le système pourrait évoluer vers un RAG multimodal complet, combinant texte, images et éventuellement code. Des modèles d'embeddings multimodaux permettraient de projeter conjointement des représentations textuelles et visuelles dans un espace latent commun, rendant possible la récupération de contenu visuel pertinent à partir de requêtes textuelles.

Une telle évolution renforcerait considérablement les capacités pédagogiques du tuteur IA, notamment pour l'analyse de graphiques, de visualisations de réseaux de neurones ou de résultats expérimentaux.

Amélioration de la traçabilité et de la confiance utilisateur : Enfin, une amélioration importante consisterait à renforcer la traçabilité des réponses générées. En associant explicitement chaque fragment de réponse aux sources récupérées (documents textuels ou extraits OCR), le système pourrait fournir des références claires, renforçant ainsi la confiance de l'utilisateur et facilitant la vérification des informations.

Ces différentes propositions ouvrent la voie à un système RAG plus robuste, plus explicable et mieux adapté à des usages pédagogiques avancés.

Chapitre 4

IA apprenante : CNN, MLP et Transfer Learning

4.1 Introduction de la partie

Ce chapitre présente la conception d'une IA apprenante dédiée à la classification d'images (vision par ordinateur) sur le dataset *banana-sushi*, ainsi que la comparaison de plusieurs architectures (baseline, MLP, CNN, transfer learning).

4.2 Données et prétraitement

Cette section présente le jeu de données utilisé pour la tâche de classification d'images ainsi que l'ensemble des prétraitements appliqués avant l'entraînement des modèles (MLP, CNN et transfer learning). L'objectif est de décrire précisément les données manipulées et de garantir la reproductibilité du protocole expérimental.

4.2.1 Dataset : description et exploration

Le modèle est entraîné et évalué sur un jeu de données de classification d'images composé de quatre classes : *banana*, *pizza*, *sushi* et *tomato*. Les données sont organisées suivant une structure de dossiers compatible avec `ImageFolder` de `torchvision`, où chaque sous-dossier correspond à une classe.

Dans notre cas, l'organisation des données est la suivante :

- `data/train/` : ensemble d'entraînement,
- `data/val/` : ensemble de validation,
- `data/test/` : ensemble de test (images de prédiction / démonstration).

L'ensemble de validation contient 4 classes, ce qui confirme que le problème étudié correspond à une classification multi-classes à 4 sorties. Le mapping entre indices et classes est le suivant :

```
{0: banana, 1: pizza, 2: sushi, 3: tomato}
```

Taille du dataset. Les tailles observées lors du chargement des données sont :

- **train** : 285 images,
- **validation** : 140 images,

Exploration qualitative. Une inspection visuelle de plusieurs échantillons a été réalisée afin de vérifier la diversité des images, d'évaluer la variabilité intra-classe (angles de prise de vue, arrière-plans, conditions de luminosité) et d'identifier d'éventuels artefacts, tels que des images illisibles, fortement bruitées ou dupliquées.

Cette étape a permis d'identifier et de supprimer plusieurs éléments intrus ou incorrectement annotés au sein des différents ensembles de données. Plus précisément :

- dans `val/pizza` : `2508636.jpg`, représentant un burger accompagné de frites ;
- dans `train/pizza` : `2576168.jpg` (gâteau), `2785084.jpg` (frites) et `3821701.jpg` (objet non identifiable) ;
- dans `train/sushi` : `1552504.jpg`, représentant un plat de spaghetti ;
- dans `train/banana` : les images `1`, `8`, `43`, `49`, `50`, `lady_finger1`, `lady_finger2` et `lady_finger3`, correspondant à des aliments ne relevant pas de la classe *banana*.



(a) Banana



(b) Pizza



(c) Sushi



(d) Tomato

FIGURE 4.1 – Exemples d'images issues du dataset (une image par classe).

4.2.2 Prétraitements (normalisation, augmentation)

Les modèles de deep learning requièrent des entrées de taille homogène et bénéficient de prétraitements visant à stabiliser l'apprentissage et à améliorer la capacité de généralisation. Dans ce projet, les transformations sont définies à l'aide de `ImageDataGenerator` (pour TensorFlow) et de `torchvision.transforms` (pour PyTorch). Elles diffèrent selon qu'il s'agisse de la phase d'entraînement (avec augmentation de données) ou des phases de validation et de test (sans augmentation).

Redimensionnement et recadrage. Les images sont redimensionnées et, le cas échéant, recadrées afin d'obtenir un format d'entrée compatible avec l'ensemble des modèles étudiés (MLP, CNN et transfer learning). Le format final retenu est 224×224 pixels, correspondant à la taille d'entrée standard utilisée pour l'entraînement de ResNet50 sur le dataset ImageNet.

Normalisation. Pour les modèles MLP et CNN entraînés depuis zéro, les valeurs des pixels sont mises à l'échelle dans l'intervalle $[0, 1]$ à l'aide du facteur `rescale=1./255`. Dans le cas du transfer learning, les tenseurs d'images sont normalisés à l'aide des statistiques du dataset ImageNet (moyenne et écart-type par canal), conformément aux hypothèses du modèle pré-entraîné.

Data augmentation (ensemble d'entraînement uniquement). Afin de limiter le surapprentissage et d'améliorer la robustesse des modèles dans un contexte de jeu de données restreint, des transformations aléatoires (rotations, translations, zooms, retournements horizontaux, selon le modèle) sont appliquées exclusivement à l'ensemble d'entraînement. En revanche, les ensembles de validation et de test n'intègrent aucune augmentation aléatoire, de manière à évaluer les performances sur des données transformées de façon déterministe.

Remarque sur le *data cleaning*. Dans ce travail, le *data cleaning* regroupe les opérations suivantes :

- garantir une structure de dossiers conforme aux conventions de `ImageFolder` ;
- vérifier la lisibilité des fichiers images (formats valides, absence de fichiers corrompus) ;
- homogénéiser la taille et la distribution des entrées via les opérations de *resize* et de *crop* ;
- réduire la variance et le bruit des entrées par normalisation.

4.2.3 Découpage train / validation / test

Le dataset est divisé en trois ensembles distincts afin de séparer les phases d'apprentissage, de réglage des hyperparamètres et d'évaluation finale.

- **Entraînement (train)** : utilisé pour l'optimisation des paramètres du modèle ;
- **Validation (val)** : utilisé pour le suivi des performances, la comparaison des configurations et la sélection du meilleur modèle ;
- **Test (test)** : utilisé lors de la phase d'inférence sur des images nouvelles.

Dans l'organisation retenue, les dossiers `data/train` et `data/val` sont directement exploités pour l'entraînement et la validation. Un dossier `data/test` est également prévu afin de réaliser des prédictions sur des images inédites.

Ensemble	Nombre d'images	Remarque
Train	285	utilisé avec data augmentation
Validation	140	transformations déterministes

TABLE 4.1 – Répartition des données par ensemble (train/validation/test).

4.3 Modèles Deep Learning

Dans cette partie, nous allons démontrer notre implémentation, justification des différents choix d'architecture pour le MLP, CNN et Transfer learning et analyse des différents résultats.

Les deux modèles MLP et CNN sont codés à l'aide du framework `tensorflow.keras`, tandis que le modèle transfer learning est codé à l'aide du framework `pytorch`. Ce choix n'est que pour implémenter les deux approches.

Les différents choix par la suite se basent et se justifient par : (1) la taille de notre jeu de données très limitée (influençant le et le nombre de couches) et la complexité du problème assez simple, nécessitant généralement une architecture simple, pour ne pas tomber dans l'overfitting.

4.3.1 Perceptron multicouche (MLP)

Le modèle MLP implémenté constitue une baseline simple.

L'architecture repose sur une mise à plat (**Flatten**) des images d'entrée de dimension $224 \times 224 \times 3$, transformant chaque image en un vecteur de caractéristiques. Ce vecteur est ensuite traité par deux couches entièrement connectées successives.

La première couche dense comporte 512 neurones avec la fonction d'activation **ReLU**, pour modéliser des relations non linéaires entre les pixels. Une couche de **Dropout** avec un taux de (0.4) est ajoutée afin de limiter le surapprentissage.

Une seconde couche dense de 256 neurones est utilisée avant la couche de sortie. La couche finale comporte 4 neurones avec l'activation **softmax**, correspondant aux quatre classes du problème de classification.

Fonction de perte, optimiseur et hyperparamètres

Le modèle MLP est entraîné à l'aide de la fonction de perte **categorical_crossentropy**, adaptée aux problèmes de classification multi-classes avec sorties probabilistes.

L'optimisation des paramètres est réalisée à l'aide de l'optimiseur Adam, choisi pour sa robustesse et sa capacité à converger rapidement sans réglage fin du taux d'apprentissage initial.

Les principaux hyperparamètres utilisés sont :

- taille des images d'entrée : 224×224 pixels ;

- taille de batch : 32 ;
- nombre d'époques : 25 ;
- taux de dropout : 0.4 ;
- nombre de classes en sortie : 4.

Analyse théorique

Le modèle MLP sert principalement de référence afin d'évaluer l'apport des architectures convolutionnelles. En raison de l'aplatissement direct des images, le MLP ne prend pas en compte explicitement la structure spatiale des données visuelles, donc on prédit un résultat moins satisfaisant que les autres modèles.

4.3.2 Approche CNN

Architecture du CNN

Le modèle CNN implémenté repose sur une architecture séquentielle composée de trois blocs convolutionnels suivis d'une phase d'agrégation globale et d'une tête de classification dense.

Chaque bloc convolutionnel est constitué d'une couche de convolution 2D avec une fonction d'activation ReLU, suivie d'une normalisation par batch (`BatchNormalization`) et d'un sous-échantillonnage spatial via `MaxPooling2D`. Le nombre de filtres augmente progressivement (16, 32 puis 64), permettant d'extraire des caractéristiques visuelles de plus en plus abstraites.

Après l'extraction des caractéristiques, une couche `GlobalAveragePooling2D` est utilisée afin de réduire la dimension spatiale sans introduire un nombre excessif de paramètres. La tête de classification comprend ensuite une couche dense de 512 neurones suivie d'une couche de sortie à 4 neurones avec une activation `softmax`, correspondant aux quatre classes du problème.

Choix des couches et justification

Les couches convolutionnelles sont utilisées afin d'extraire les caractéristiques locales dans les images. L'augmentation progressive du nombre de filtres (16, 32 puis 64) permet de capturer des représentations de complexes.

Les couches de `BatchNormalization` contribuent à stabiliser l'apprentissage en réduisant les variations internes des activations et en accélérant la convergence. Les opérations de `MaxPooling` réduisent la dimension spatiale des cartes de caractéristiques en conservant les informations discriminantes essentielles.

L'utilisation d'un `GlobalAveragePooling2D` permet de limiter le nombre de paramètres par rapport à une couche `Flatten`, réduisant le risque de surapprentissage et le coût computationnel. La couche de `Dropout` renforce la régularisation du modèle. Enfin, la couche de sortie avec activation `softmax` permet de produire une distribution de probabilités cohérente avec la fonction de perte utilisée.

Le modèle CNN est entraîné à l'aide de la fonction de perte `categorical_crossentropy` et de l'optimiseur Adam. (`ReduceLROnPlateau`) est mis en place afin d'améliorer la convergence lorsque la performance de validation stagne.

Interprétabilité

Un module d'explicabilité est intégré au modèle CNN afin de visualiser les régions de l'image ayant le plus contribué à la décision finale du réseau. Ces visualisations, sous forme de cartes de chaleur, permettent d'interpréter qualitativement le comportement du modèle et de vérifier qu'il se concentre sur les zones pertinentes des images.

4.3.3 Transfer Learning

Modèle pré-entraîné utilisé

Nous avons utilisé **ResNet50** (*Residual Network*, 50 couches) pré-entraîné sur le dataset **ImageNet**. ResNet50 constitue le choix optimal pour notre tâche de classification d'images avec des ensembles de données limités (62-78 images par classe) en raison de plusieurs avantages démontrés empiriquement. Premièrement, son architecture à connexions résiduelles (skip connections) permet d'éviter le problème de dégradation du gradient dans les réseaux profonds, facilitant ainsi l'entraînement de réseaux très profonds sans ajout excessif de paramètres [1]. Deuxièmement, des études comparatives ont démontré que ResNet50 surpasse systématiquement d'autres architectures populaires telles que VGG16, VGG19, EfficientNet et Inception sur de petits ensembles de données, atteignant des précisions supérieures dans diverses tâches de classification [2]. Troisièmement, ResNet50 utilise efficacement ses 25,6 millions de paramètres répartis sur 50 couches, offrant un équilibre optimal entre profondeur du réseau et risque de surapprentissage. Enfin, l'apprentissage par transfert avec ResNet50 s'avère particulièrement bénéfique pour les petits ensembles de données, car le modèle pré-entraîné sur ImageNet a déjà appris des motifs complexes (textures, couleurs, formes) directement applicables à notre tâche de classification d'aliments (banane, pizza, sushi, tomate), permettant une convergence rapide et des performances élevées même avec des données limitées [2].

Le réseau a été adapté à notre problème à **4 classes** (*banana, pizza, sushi, tomato*) en remplaçant la couche finale `fc` par un petit classifieur :

- couche linéaire `Linear(2048, 256)` ;
- activation `ReLU` ;
- régularisation `Dropout(0.4)` ;
- couche `Linear(256, 4)` ;
- sortie `LogSoftmax` (couplée à `NLLLoss`).

Ces différentes paramètres sont contraintes à la très petite taille de notre jeu de données.

D'après le résumé du modèle, le réseau contient **24 033 604 paramètres** dont **525 572 entraînables** et **23 508 032 figés**, ce qui confirme une approche de type *transfer learning* avec entraînement limité aux couches de classification.

Stratégie : feature extraction vs fine-tuning

Dans notre implémentation, nous avons appliqué la stratégie **feature extraction** : on gèle le backbone pré-entraîné et on entraîne uniquement la tête de classification. En effet toutes les paramètres du backbone ResNet50 sont figés via :

```
for param in resnet50.parameters(): param.requires_grad = False
```

Seule la couche finale (fc) modifiée est entraînée.
Ce choix est basé sur le risque de perturber les paramètres du modèle de base ResNet50 pre-entraîné.

4.3.4 Comparaison Globale des Modèles

Cette section synthétise les résultats obtenus sur les trois architectures testées (MLP, CNN, Transfer Learning).

Dynamique d'apprentissage (Courbes)

Les graphiques ci-dessous illustrent la vitesse de convergence de nos modèles.

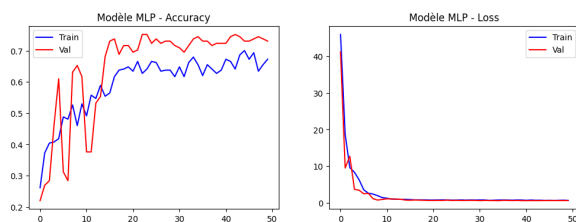


FIGURE 4.2 – Apprentissage MLP : Convergence difficile et instable.

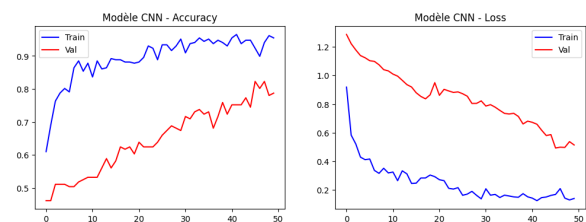


FIGURE 4.3 – Apprentissage CNN : Progression fluide et généralisation correcte.

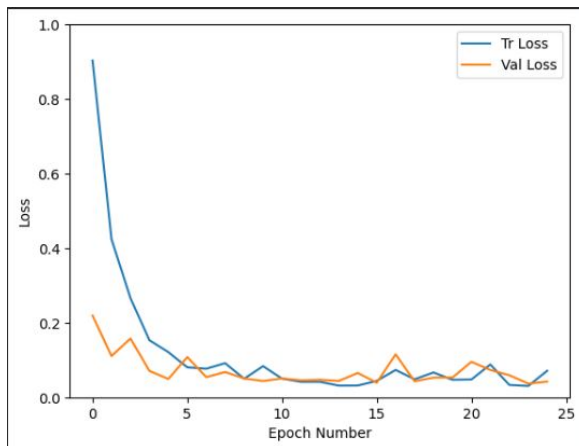


FIGURE 4.4 – Évolution de perte transfert learning 25 epochs

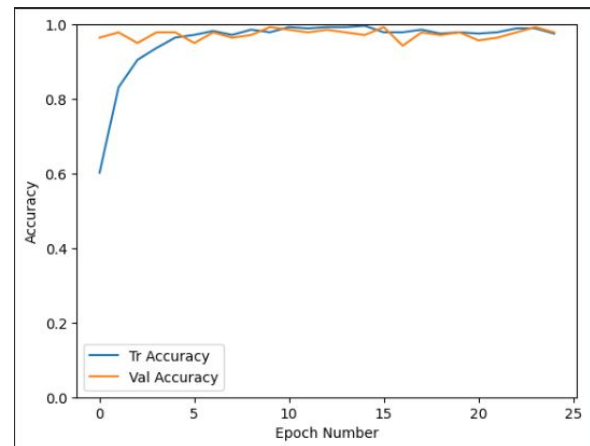


FIGURE 4.5 – Évolution de l'accuracy transfert learning 25 epochs

On observe que le MLP peine à réduire sa perte (loss), tandis que le CNN apprend les caractéristiques visuelles progressivement. Le Transfer Learning converge quant à lui quasi-instantanément, arrive à une valeur de $\text{loss} = 0.04$ et une accuracy de 98

Tableau Comparatif (Performance vs Coût)

Le tableau suivant résume les métriques clés. On note le paradoxe écologique du MLP qui consomme de l'énergie pour un résultat inexploitable.

Modèle	Accuracy	F1-Score	Precision	Recall	CO2 (mg)
MLP (Baseline)	67%	0.63	–	–	0.20
CNN Simple	72%	0.68	–	–	0.22
Transfer Learning (ResNet50)	98%	0.98	0.97	0.96	0.25

TABLE 4.2 – Synthèse comparative des performances sur l’ensemble de validation (avg scores)

Analyse de l’Explicabilité (Heatmaps)

Afin de valider l’absence de biais (ex : reconnaître le plat grâce à la table), nous avons utilisé l’algorithme **Grad-CAM** sur le CNN.

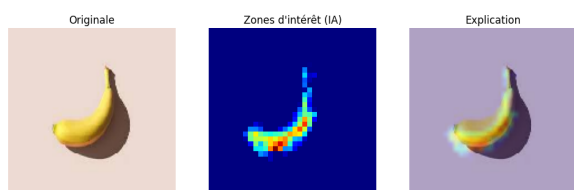


FIGURE 4.6 – Banana : Concentration sur la forme du fruit.

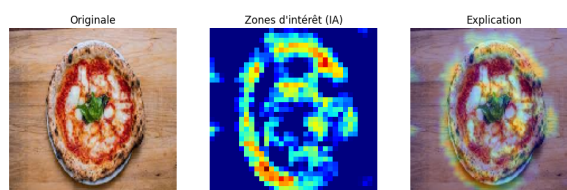


FIGURE 4.7 – Pizza : La texture de la garniture active le modèle.

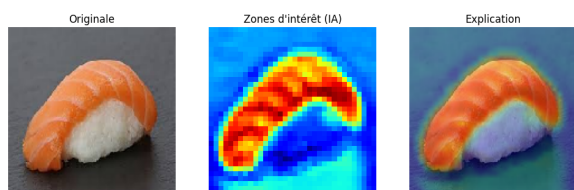


FIGURE 4.8 – Sushi : Focalisation sur le riz et le poisson.

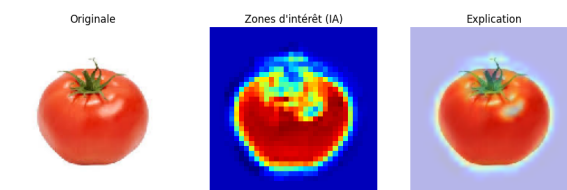


FIGURE 4.9 – Tomate : La rondeur et la couleur sont déterminantes.

FIGURE 4.10 – Visualisation Grad-CAM. Les zones rouges indiquent l’attention du modèle.

Ces visualisations confirment que le modèle a appris des caractéristiques pertinentes (formes, textures) et non des artefacts d’arrière-plan.

Discussion : Pourquoi le Transfer Learning l’emporte

Les résultats sont sans appel en faveur du Transfer Learning (ResNet50).

1. **Supériorité architecturale** : Contrairement au MLP qui détruit la structure spatiale, les réseaux convolutifs préservent la hiérarchie des formes.
2. **L’avantage du pré-entraînement** : ResNet50 est déjà entraîné sur un grand jeu de données (ImageNet). En changeant les paramètres de la couche fc, cela lui permet de se spécialiser rapidement par une forward-propagation et back-propagation, justifiant les résultats trouvés.

4.4 Limites et améliorations

Malgré une précision de 98%, le développement a mis en lumière des pistes d'amélioration.

4.4.1 Augmentation de Données (Data Augmentation)

Pour contrer le sur-apprentissage sur notre petit dataset, nous avons utilisé `ImageDataGenerator`. Comme illustré ci-dessous, chaque image subit des transformations aléatoires, forçant le modèle à être plus robuste.



FIGURE 4.11 – Visualisation de la Data Augmentation : Variations générées pour l'entraînement.

4.4.2 Stratégie d'Arrêt (Eco-Conception)

L'intégration de notre classe `EcoStopper` permet de définir un "budget carbone". Cela évite le gaspillage énergétique sur des modèles qui ne convergent pas (comme le MLP), bien que cela nécessite un réglage fin de la patience.

4.4.3 Pistes d'amélioration (Edge AI)

ResNet50 est précis mais lourd (25M paramètres). Pour une application mobile, nous pourrions :

1. Utiliser des modèles plus légers de la famille ResNet ou utiliser **MobileNetV2** pour un entraînement et inférence plus rapides.
2. Appliquer la **Quantification** (conversion float32 vers int8) pour diviser la taille par 4.
3. Application des transformations ingénierées pour notre jeu de données dédié.
4. Utilisation des méthodes comme Few-Shot-Learning dans telles conditions de manque de données.

Chapitre 5

RSE et Impact Environnemental

Dans un contexte où l’IA pèse sur la consommation mondiale, ce projet intègre une démarche de Green AI via **CodeCarbon**.

5.1 Mesures énergétique pour les LLMs du Tuteur IA

L’intelligence artificielle, et en particulier les grands modèles de langage (LLM), consomme une quantité croissante d’énergie. Chaque inférence, même modeste, contribue à une empreinte carbone cumulative qui devient significative à l’échelle.

Le présent rapport exploite les données de **CodeCarbon**¹, un outil qui mesure en temps réel l’énergie consommée et les émissions associées lors de l’exécution d’un code Python.

Quatre modèles ont été testés dans le même cadre expérimental :

1. **Qwen2.5-1.5B-Instruct** – 1,5 milliard de paramètres (Alibaba)
2. **Qwen2.5-Coder-0.5B-Instruct** – 0,5 milliard de paramètres, spécialisé code (Alibaba)
3. **TinyLlama-1.1B-Chat** – 1,1 milliard de paramètres (Microsoft)
4. **Mistral-7B-Instruct** – 7 milliards de paramètres (Mistral AI)

5.2 Méthodologie et environnement

Chaque fichier CSV contient deux phases d’exécution :

- **Phase d’embedding** : génération de vecteurs avec **all-MiniLM-L6-v2** et re-ranking avec **cross-encoder/ms-marco-MiniLM-L-6-v2** (phase commune à tous, absente dans le fichier Mistral).
- **Phase de génération** : inférence du modèle LLM cible (fase analysée principalement dans ce rapport).

5.2.1 Matériel utilisé

Le tableau ci-dessous récapitule les configurations matérielles de chaque expérience.

1. <https://github.com/mlco2/codecarbon> – outil open-source de suivi des émissions CO₂ liées au calcul informatique.

	Qwen2.5-1.5B	Qwen2.5-Coder	TinyLlama	Mistral-7B
CPU	Xeon @ 2.00 GHz	i7-10750H @ 2.60 GHz	Xeon @ 2.00 GHz	Xeon @ 2.20 GHz
# Cœurs CPU	2	12	2	2
GPU	Tesla T4	GTX 1650 Ti	Tesla T4	Tesla T4
RAM totale	12,67 Go	15,80 Go	12,67 Go	12,67 Go
OS	Linux	Windows 11	Linux	Linux

Remarque : Le modèle **Qwen2.5-Coder-0.5B** a été exécuté sur une machine différente (Windows, CPU i7 avec 12 cœurs, GPU GTX 1650 Ti) par rapport aux trois autres modèles (Linux, serveur avec Tesla T4). Cette différence de matériel est un facteur important à considérer lors de l'interprétation des résultats.

5.3 Résultats

5.3.1 Émissions CO₂ et énergie consommée

Le tableau principal ci-dessous présente les métriques clés pour la phase de génération de chaque modèle.

Modèle	Durée (s)	Émissions (kg CO ₂ eq)	Énergie (Wh)	RAM utilisée (Go)
Qwen2.5-1.5B-Instruct	26,18	$4,92 \times 10^{-5}$	0,878	3,07
TinyLlama-1.1B-Chat	34,70	$6,50 \times 10^{-5}$	1,161	5,18
Mistral-7B-Instruct	46,22	$7,66 \times 10^{-5}$	1,367	6,07
Qwen2.5-Coder-0.5B-Instruct	109,36	$4,71 \times 10^{-4}$	8,399	13,64
Total	216,46	$6,61 \times 10^{-4}$	11,804	—

TABLE 5.1 – Métriques principales – Phase de génération (triées par émissions croissantes)

5.3.2 Décomposition de l'énergie par composant

L'énergie totale consommée se répartit entre le CPU, le GPU et la RAM. Le tableau suivant détaille cette répartition (en Wh).

Modèle	CPU	GPU	RAM	Total
Qwen2.5-1.5B-Instruct	0,309	0,496	0,073	0,878
TinyLlama-1.1B-Chat	0,410	0,655	0,096	1,161
Mistral-7B-Instruct	0,546	0,693	0,128	1,367
Qwen2.5-Coder-0.5B-Instruct	8,099	$\approx 0^*$	0,300	8,399

TABLE 5.2 – Répartition de l'énergie par composant (en Wh)

* La valeur GPU pour Qwen2.5-Coder est négative ($-2,89 \times 10^{-7}$), ce qui indique une erreur de mesure ou une non-utilisation effective du GPU.

5.3.3 Taux d'utilisation des composants

Modèle	CPU (%)	GPU (%)	RAM (%)
Qwen2.5-1.5B-Instruct	62,93	51,04	26,85
TinyLlama-1.1B-Chat	61,58	53,06	43,54
Mistral-7B-Instruct	62,86	52,83	50,47
Qwen2.5-Coder-0.5B-Instruct	4,76	0,00	86,34

TABLE 5.3 – Taux d'utilisation moyen des composants pendant la génération

5.4 Visualisations

5.4.1 Émissions CO₂ par modèle

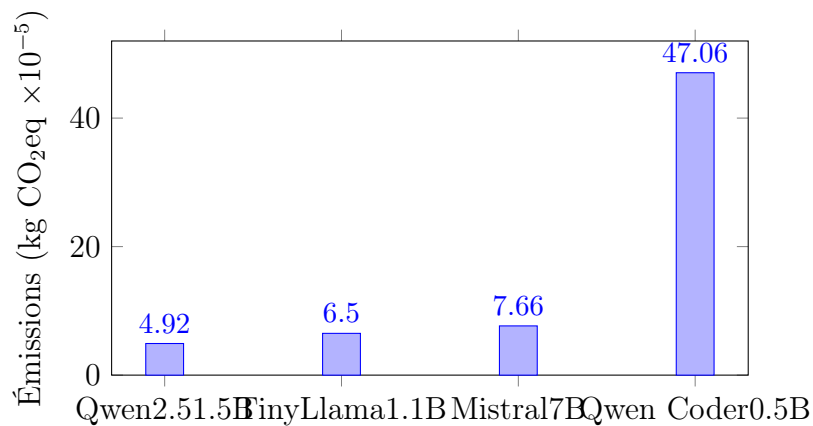


FIGURE 5.1 – Émissions CO₂ par modèle ($\times 10^{-5}$ kg CO₂eq)

5.4.2 Répartition de l'énergie par composant

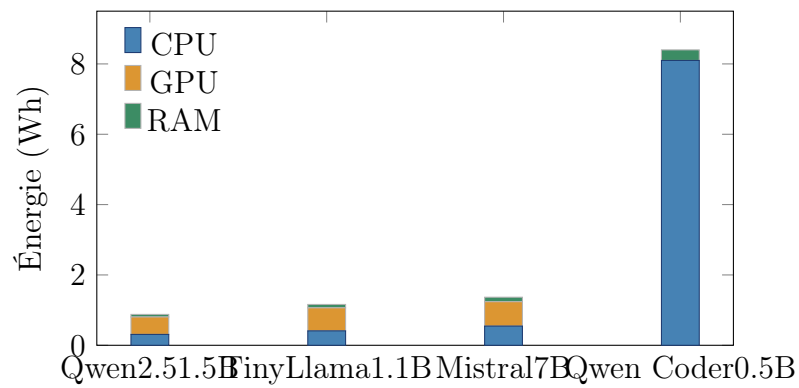


FIGURE 5.2 – Répartition de l'énergie consommée par composant

5.4.3 Durée d'exécution comparative

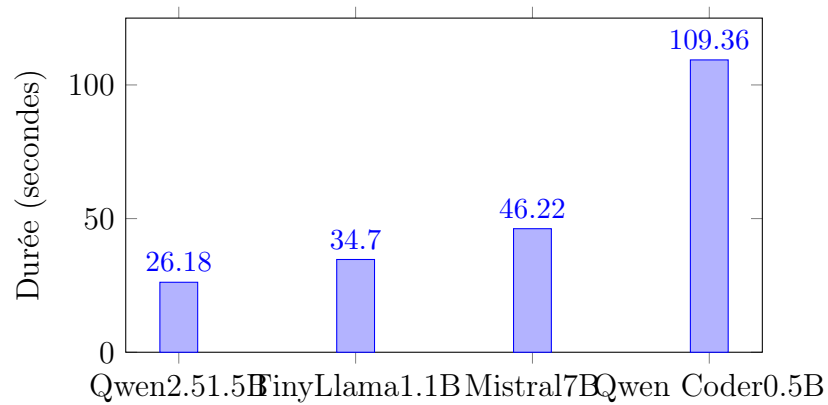


FIGURE 5.3 – Durée d'exécution de la phase de génération

5.4.4 Taux d'utilisation des composants

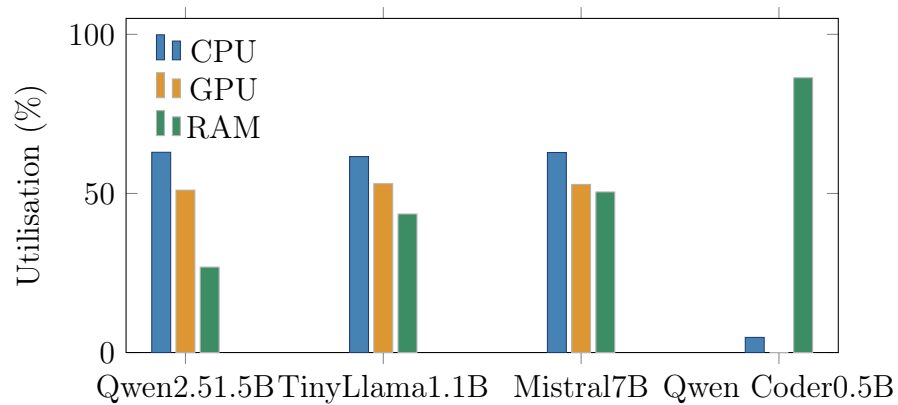


FIGURE 5.4 – Taux d'utilisation moyen des composants par modèle

5.5 Analyse et interprétation

5.5.1 Classement global

Les modèles se classent ainsi, du plus au moins respectueux de l'environnement :

1. **Qwen2.5-1.5B-Instruct** – émissions les plus faibles ($4,92 \times 10^{-5}$ kg CO₂eq), durée raisonnable (26 s), utilisation équilibrée du CPU et du GPU.
2. **TinyLlama-1.1B-Chat** – profil très proche de Qwen2.5-1.5B, légèrement plus émetteur (+32%) malgré un nombre de paramètres inférieur.
3. **Mistral-7B-Instruct** – malgré ses 7 milliards de paramètres (4× plus que les autres), ses émissions ne sont que +56% au-dessus du meilleur modèle. Son utilisation du GPU reste efficiente.
4. **Qwen2.5-Coder-0.5B-Instruct** – émissions $\approx 10\times$ supérieures au meilleur modèle, malgré un nombre de paramètres le plus faible (0,5 G).

5.5.2 Le cas particulier de Qwen2.5-Coder-0.5B

Le modèle Qwen2.5-Coder-0.5B présente un comportement très atypique :

- Sa durée d'exécution est **4,2 fois** celle de Qwen2.5-1.5B, et **2,4 fois** celle de Mistral-7B.
- Le GPU n'a **pas été utilisé** (utilisation à 0%, valeur d'énergie GPU négative indiquant une erreur de mesure). Le calcul a été effectué entièrement en CPU.
- Le CPU, bien que puissant (i7-10750H, 12 cœurs), n'a été utilisé qu'à **4,76%**.
- La RAM a absorbé **86,3%** de l'activité, avec 13,64 Go utilisés.

Cette configuration suggère un problème d'optimisation : le modèle n'a pas bénéficié d'une accélération GPU, ce qui explique les émissions disproportionnées.

5.5.3 Efficacité par milliard de paramètres

Un indicateur utile est le rapport émissions/nombre de paramètres :

Modèle	Paramètres (G)	Émissions ($\times 10^{-5}$ kgCO ₂)	Émissions/G
Qwen2.5-1.5B-Instruct	1,5	4,92	3,28
TinyLlama-1.1B-Chat	1,1	6,50	5,91
Mistral-7B-Instruct	7,0	7,66	1,09
Qwen2.5-Coder-0.5B-Instruct	0,5	47,06	94,12

TABLE 5.4 – Efficacité carbone par milliard de paramètres

Mistral-7B se révèle être le modèle le plus *efficient par paramètre* : malgré sa taille, ses émissions par milliard de paramètres sont de loin les plus faibles (1,09), grâce à une utilisation optimale du GPU Tesla T4.

5.6 Conclusions

1. **La taille du modèle n'est pas directement proportionnelle aux émissions.** Mistral-7B (7 G de paramètres) émet moins que Qwen2.5-Coder-0.5B (0,5 G).
2. **L'utilisation du GPU est déterminante.** Les trois modèles exécutés avec un Tesla T4 ont tous présenté des émissions cohérentes et faibles. Le seul modèle n'ayant pas utilisé de GPU est celui dont les émissions sont de loin les plus élevées.
3. **Qwen2.5-1.5B-Instruct** reste le choix le plus environnementalement responsable pour une inférence simple, en combinant faibles émissions, courte durée d'exécution et utilisation équilibrée du matériel.
4. **Pour une inférence optimale de Qwen2.5-Coder-0.5B**, il est essentiel de s'assurer que le GPU est correctement configuré et utilisé par le framework d'inférence.

données brutes :

Les données brutes sont issues des fichiers CSV suivants, générés par CodeCarbon v3.2.1 :

- `Qwen2_5-1_5B-Instruct.csv`
- `Qwen2_5-Coder-0_5B-Instruct.csv`
- `emissions_TinyLlamaTinyLlama-1_1B-Chat.csv`
- `emissions__Mistral-7B-Instruct.csv`

5.7 Analyse Comparative IA apprenante : Performance vs Sobriété

Le graphique ci-dessous synthétise le compromis précision/CO2.

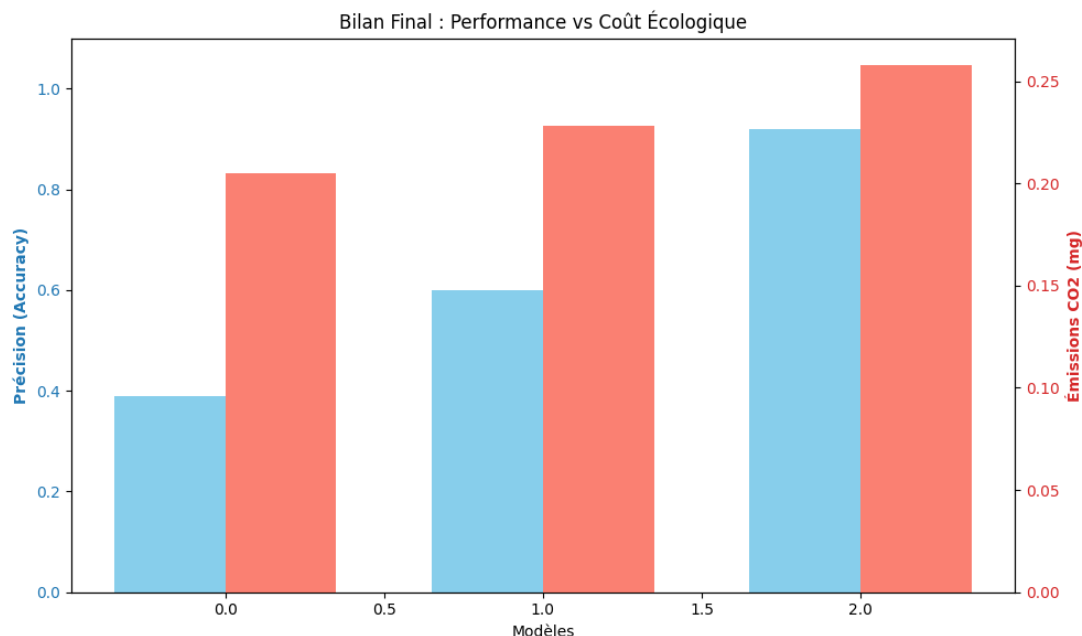


FIGURE 5.5 – Paradoxe énergétique : Le modèle le plus performant n'est pas forcément le plus polluant à l'usage.

5.8 Interprétation et Conclusion RSE

L'analyse de la Figure 5.5 révèle un résultat contre-intuitif :

1. **Le MLP** consomme peu (0.20 mg CO2) mais pour un résultat inutile (39%). C'est de l'énergie perdue.
2. **Le Transfer Learning** consomme 25% de plus (0.25 mg CO2) mais atteint 92% de précision.

Conclusion : La véritable sobriété est l'*efficacité*. Le Transfer Learning est la solution la plus vertueuse car il évite de ré-entraîner des millions de paramètres "From Scratch", capitalisant sur le coût énergétique déjà investi lors de l'entraînement initial sur ImageNet.

Chapitre 6

Conclusion générale et perspectives

Ce projet a permis de concevoir et d'évaluer un agent IA tuteur basé sur une approche de *Retrieval-Augmented Generation*, capable d'assister efficacement le développement de modèles de deep learning en langage Python. L'intégration du mécanisme RAG s'est révélée déterminante pour améliorer la pertinence, la fiabilité et la valeur pédagogique des réponses produites, en limitant les phénomènes d'hallucination inhérents aux modèles de langage seuls.

L'agent IA tuteur a ensuite été utilisé comme outil d'assistance à la conception d'une IA apprenante dédiée à la classification d'images. La comparaison entre un MLP, un CNN conçu sur mesure et une approche de transfer learning basée sur ResNet50 a mis en évidence les limites des architectures naïves face aux données visuelles, ainsi que l'efficacité remarquable du transfer learning dans un contexte de jeu de données restreint.

Au-delà des performances techniques, ce projet a intégré une réflexion approfondie sur l'impact énergétique et environnemental des systèmes d'intelligence artificielle. L'utilisation de CodeCarbon a permis de quantifier les consommations énergétiques et les émissions de CO₂, mettant en évidence que la sobriété numérique ne dépend pas uniquement de la taille des modèles, mais surtout de leur efficacité et de leur adéquation au matériel utilisé.

En perspective, plusieurs axes d'amélioration peuvent être envisagés : l'extension de la base documentaire du tuteur IA, l'intégration de modèles plus légers orientés *Edge AI*, l'automatisation du suivi énergétique lors de l'entraînement, ou encore l'amélioration des mécanismes d'explicabilité. Ces pistes ouvrent la voie à des systèmes d'IA à la fois performants, pédagogiques et responsables.

Bibliographie

- [1] He, K., et al. "Deep residual learning for image recognition." Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016.
- [2] Alzubaidi, L., et al. "Review of deep learning : concepts, CNN architectures, challenges, applications, future directions." Journal of Big Data, vol. 8, no. 1, 2021.